Ders 25 Çalışma Özeti

Ders 25 Çalışma Özeti

Genel Konular

- Ara sınavda en sık görülen hatalar (genel geri bildirim)
  - Sütun silme sorusunda sütunun yalnızca sıfırlanması ama kaydırılmaması ya da kaydırılıp son sütunun sıfırlanmaması.
  - Derste gösterilen akış diyagramı notasyonunun dışında, kişisel şekillerle çizim yapılması. Bundan sonraki sınavlarda derste gösterilen notasyon dışındaki çizimler kabul edilmeyecek.
  - `pow` fonksiyonu kullanılmaması istendiği hâlde kullanılması. Üs hesabı döngüyle yapılmalıdır.
  - For kullanılması gereken yerde while, while kullanılması gereken yerde for kullanılması.
- Dönem projesi: Kızma Birader oyunu
  - Hint kökenli bir oyundur. Ülkeye göre farklı versiyonları ve kuralları vardır. Projede proje dokümanındaki temel kurallar esas alınır.
  - Temel kurallar: her oyuncunun 4 piyonu vardır. 2, 3 veya 4 oyuncuyla oynanır. Her turda 1-6 arası tek bir zar atılır. Piyon başlangıç alanından ancak 6 gelince çıkabilir. 6 gelen oyuncu ya yeni bir piyonu oyuna sokar ya da tahtadaki bir piyonunu 6 ilerletir. Her hücrede yalnızca bir piyon bulunabilir. Yoldaki piyonların üzerinden geçilebilir. Zar bir rakip piyonun bulunduğu hücreye getirirse rakip piyon başlangıca döner. Oyuncunun kendi piyonunun bulunduğu hücreye getirirse o hamle yapılamaz. Hedefe ulaşan piyonlara dokunulmaz. 4 piyonunu ilk hedefe ulaştıran kazanır ve oyun biter.
  - Grafik arayüz istenmez. Program menüler ve komutlar üzerinden çalışır. Tahta her hamleden sonra ekranda gösterilmelidir. Önceki pozisyonun da saklanması, hamlenin doğruluğunun kontrolünü kolaylaştırır.
  - Opsiyonel kurallar (iki zar, güvenli bölgeler, başa döndüren hücreler gibi) eklenebilir, ama temel kurallarla çelişmemelidir. Örneğin iki zar kullanılsa bile evden çıkış yine 6 ile olmalıdır.
  - Akış diyagramı istenmez. Kütüphane sınırlaması yoktur. Tek bir kaynak dosya (source file) olmalıdır, başlık dosyaları (header) kullanılabilir. Zar için rastgele sayı üreten fonksiyon (random) kullanılır.
  - Kullanıcının açık hatalı girişleri (5 oyuncu, zarın 7 gelmesi gibi) kontrol edilmelidir.
- Proje için tasarım önerileri
  - Tahtadaki rotayı her köşede yön değiştiren çok sayıda if ile kontrol etmek yerine, rota tek uzun bir yol olarak düşünülebilir. Her adımın matristeki satır-sütun karşılığı bir eşleme (mapping) tablosunda saklanır. Örnek: 1. adım (4,1), 2. adım (4,2), 3. adım (4,3), 4. adım (4,4), 5. adım (3,4)…
  - Her rengin başlangıç noktası farklıdır. Bu tasarımda dikkate alınmalıdır.
  - Tek bir matris hücresine birden fazla karakter yazılamadığı için hücrelerdeki etiketler (ör. “Home Yellow 1”) için birkaç hücre birleştirilebilir ya da sayısal kodlar kullanılabilir. Seçilen yöntem teslim edilen dokümanda açıklanmalıdır.
  - Modüler programlama: büyük problemi alt problemlere bölmek çözümü kolaylaştırır. Zar atma, oyuncu sayısını seçme, menüyü gösterme, gelen zara göre ne yapılacağına karar verme, piyonun evde mi tahtada mı olduğunu tutma ayrı modüllerdir. C’de modüller fonksiyonlara karşılık gelir. Fonksiyon yazmak zorunlu değildir ama kabul edilir.

  - Okunabilir kod: amacına yönelik değişken adları, açıklama satırları, düzgün hizalama ve boşluklar. Fonksiyon kullanılmayacaksa hangi bloğun ne yaptığını açıklama satırlarıyla belirtmek daha da önemlidir.
- Canlı kodlama 1: iki kelimenin sözlükteki sırası (ekrandan, 01:15:25)
  - İki kelime `scanf("%s", ...)` ile okunur. `%s` ile okurken dizi adının önüne `&` konmaz, çünkü dizinin adı ilk gözünün adresidir. `&kelime[0]` yazmak da aynı anlama gelir.
  - Aynı tipteki değişkenler virgülle aynı satırda tanımlanabilir: `char kelime[100], kelime2[100];`
  - Döngü: iki kelimede de null karakterine gelinmediği ve harfler eşit olduğu sürece `i` artırılır.
  - Döngüden sonra `kelime[i] < kelime2[i]` ise birinci kelime önce gelir. Eşitse kelimeler aynıdır. Değilse ikinci kelime önce gelir.
  - Kısa kelime uzun kelimenin başıysa (Ali - Alibaba) null karakteri her harften küçük olduğu için kısa kelime önce gelir.
  - Sonuç mesajında kelimelerin kendisi `%s` ile yazdırılabilir: `printf("%s kelimesi sozlukte %s kelimesinden once gelir", kelime, kelime2);`
- Canlı kodlama 2: kelimelerin sütunlarını yazdırma (ekrandan, 01:40:05)
  - Kelimeler bir karakter matrisinin satırlarına okunur: `scanf("%s", dictionary[i])` ya da `scanf("%s", &dictionary[i][0])`. Bir sütuna doğrudan kelime okunamaz.
  - Önce en uzun kelimenin uzunluğu (`max`) bulunur. Her satırda null karakterine kadar sayılır.
  - Sonra `j ← 0 .. max−1` dış döngüde, `i ← 0 .. kelimeSayisi−1` iç döngüde sütunlar yukarıdan aşağıya yazdırılır. Her sütundan sonra `\n`.
  - Örnek: `ali`, `sel`, `ilk` için sütunlar `asi`, `lel`, `ilk` olur.

Hocanın Özellikle Vurguladığı Kısımlar

- Kodda tek bir işlem olsa bile döngü ve if bloklarını süslü parantez içine alın. Bu derste uzunluk hesaplayan for'un süslü parantezi unutulduğu için `max` yanlış hesaplandı.
- `NULL` ile karakter karşılaştırması bazı derleyicilerde "comparison between pointer and integer" uyarısı verir. Bu uyarı bir hata değildir, ama karakteri `'\0'` veya `0` ile karşılaştırmak daha doğrudur. Koşulda yalnızca `kelime2[i]` yazmak da aynı anlama gelir. `NULL` tanımlı olmayan ortamlar olabilir. Gerekirse `#undef NULL` ve `#define NULL 0` ile tanımlanabilir.
- Uzunluğa bakarak iki kelimenin sözlükteki sırası belirlenemez. Karşılaştırma harf harf yapılmalıdır.
- Hoca kodu yazarken değişkenleri en sonda yukarı çıkıp tanımlıyor. Öğrenciler de böyle çalışabilir.
- `\t` bir sekme kadar boşluk bırakır. `%3d`, `%5.2f` gibi formatlı yazımlarla çıktı hizalanabilir.
- Dönem projesine erken başlayın. Önce tasarımı yapın, sonra kodlamaya geçin. Önce temel kuralları (ör. tek piyonun hareketi), sonra birden fazla piyonu uygulayın.
- Final için: derste çözülen soruları kapatarak kendi başınıza yeniden çözün. Genel kültür ve kuram soruları da gelecek. İki tabanında toplama-çıkarma, ikiye tümleyen ve işletim sisteminin görevleri gibi konuları bilmeniz gerekir.

Kısa Tekrar Notları

- `scanf("%s", kelime)` — dizi adına `&` konmaz (`&kelime[0]` de olur).
- Kelime karşılaştırma: `while(k1[i] != '\0' && k2[i] != '\0' && k1[i] == k2[i]) i++;` sonra `k1[i]` ile `k2[i]` karşılaştırılır.
- Kelimeyi matrisin satırına okumak: `scanf("%s", dictionary[i])`.
- En uzun kelime: her satırda `'\0'`'a kadar say, en büyüğü tut.
- Sütun yazdırma: dış döngü sütun (`j`), iç döngü satır (`i`).
- Tek satırlık bloklarda da süslü parantez kullanın.
- Proje: tek kaynak dosya, menü tabanlı, her hamlede tahta gösterilir, rota için eşleme tablosu düşünün.

Detaylı Açıklamalar

Dersin ilk bölümünde ara sınav sonuçları ve dönem projesi konuşulur. Hoca, proje dokümanındaki kuralları ve öğrencilerin sorularını tek tek cevaplar. Aradan sonra OnlineGDB üzerinde öğrencilerle birlikte iki küçük C programı yazılır. Hoca kodu bilinçli olarak hızlı yazar ve hataların birlikte bulunmasını sağlar.

Sütun yazdırma programında bulunan hatalar

İlk çalıştırmada hiçbir çıktı alınmadı. Sırasıyla şu hatalar bulundu:

1. Yazdırma döngüsünde `i` ile `j`'nin yerleri karışmıştı. `dictionary[i][j]` yazılırken `i` satırı (kelimeyi), `j` sütunu göstermelidir.
2. Dış döngü sınırları da buna göre düzeltildi: dış döngü `j < max`, iç döngü `i < kelimeSayisi`.
3. Uzunluğu hesaplayan for döngüsünün gövdesinde birden fazla satır olduğu hâlde süslü parantez yoktu. Bu yüzden `max` yanlış hesaplandı. Hata, kontrol amacıyla `max` ekrana yazdırılarak bulundu.
4. Bir öğrenci `max`'tan 1 çıkarılması gerektiğini düşündü. Döngü koşulu `j < max` olduğu için `max` uzunluğun kendisi olmalıdır. 3 harfli kelimede `max = 3` olmalı ve döngü 0, 1, 2 için çalışmalıdır.

Düzeltmelerden sonra `ali`, `sel`, `ilk` için doğru çıktı alındı.

Açık kalan sorun: kelimenin sonundan sonrasını okumak

Kelimeler farklı uzunluktaysa (ör. `ali`, `mehmet`, `sureyya`, `veli`) yazdırma döngüsü kısa kelimelerin null karakterinden sonraki gözlerine de bakar. Programda yalnızca `'a'` ile `'z'` arasındaki karakterler yazdırılıyordu. Ama bu gözlerde tesadüfen bir harf varsa o da yazdırılır. Hoca bu sorunu bir sonraki derse bıraktı. Sınıfta önerilen çözümler:

- Matrisin bütün gözlerine başlangıçta 0 (null) atamak.
- Her kelimenin uzunluğunu saklamak (ör. matrisin ilk gözünde ya da ayrı bir dizide) ve `j` uzunluktan küçükse yazdırmak.

Ders sonrasında birinci yöntemle (`char dictionary[50][50] = {{0}};`) kod derlendi ve denendi. `ali mehmet sureyya veli` için sütunlar `amsv`, `leue`, `ihrl`, `mei`, `ey`, `ty`, `a` olarak doğru yazdırıldı.

```c
#include <stdio.h>

int main()
{
    char dictionary[50][50]={{0}};      /* bütün gözler '\0' */
    int i,j,kelimeSayisi,max;

    scanf("%d",&kelimeSayisi);
    for(i=0;i<kelimeSayisi;i++)
        scanf("%s",dictionary[i]);

    max=0;
    for(i=0;i<kelimeSayisi;i++)
    {
        j=0;
        while(dictionary[i][j]!='\0')
            j++;
        if(j>max)
            max=j;
    }

    for(j=0;j<max;j++)
    {
        for(i=0;i<kelimeSayisi;i++)
```

```
            if(dictionary[i][j]!='\0')
                printf("%c",dictionary[i][j]);
        printf("\n");
    }
    return 0;
}
```

Kelime karşılaştırma programı da `NULL` yerine `'\0'` ile yazılıp derlendi. `ali-ahmet`, `ali-alibaba` ve `ali-ali` için doğru sonuçlar verdi.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.